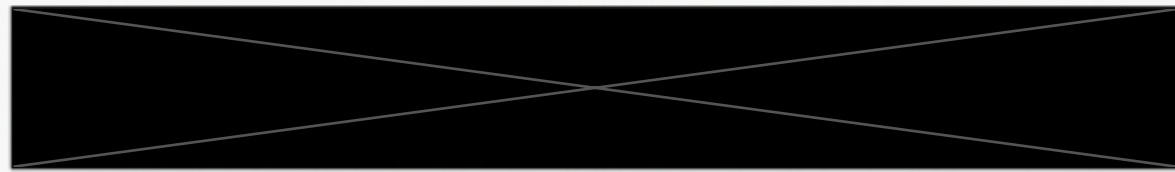
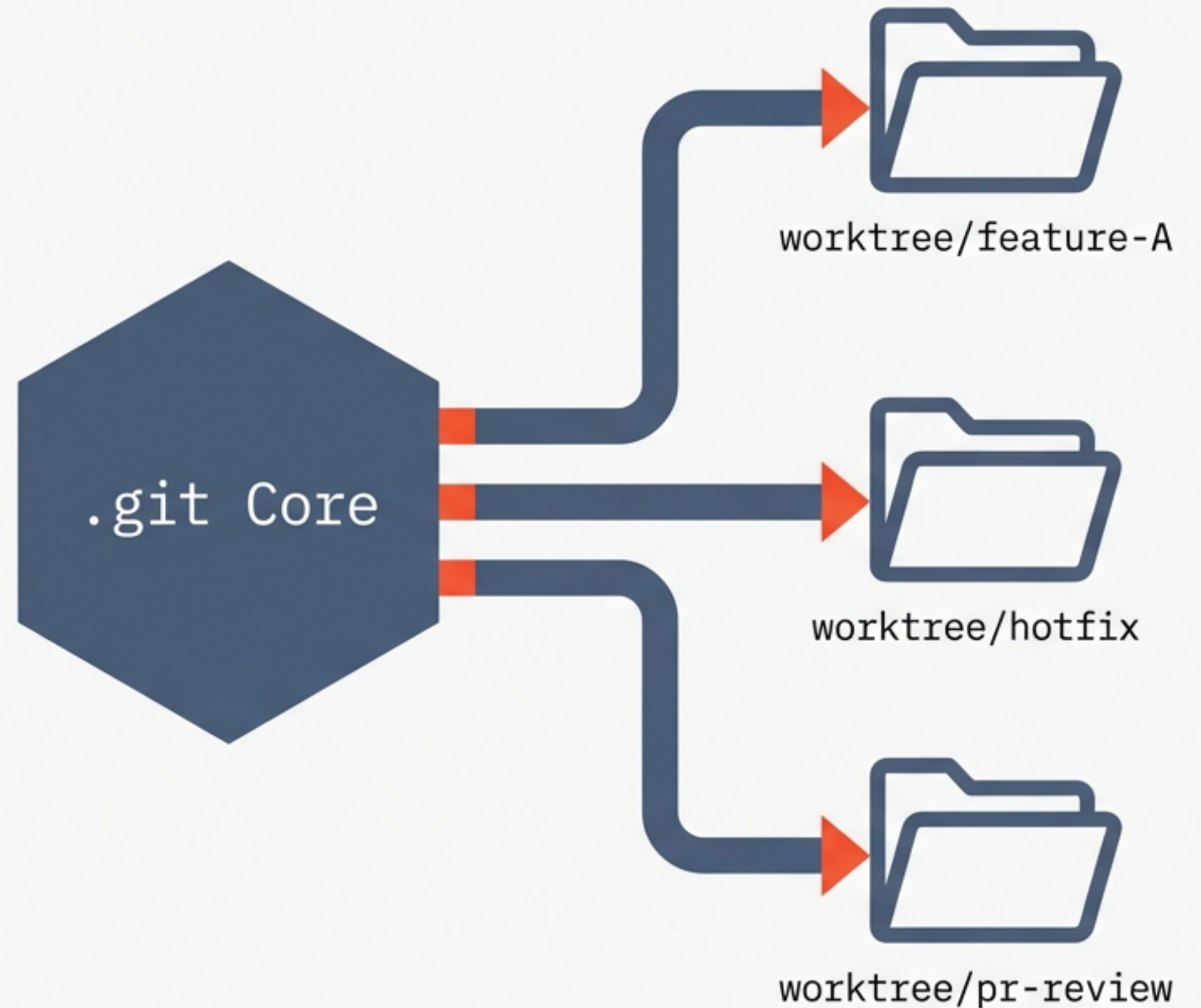


Git Worktrees: Multiple Working Directories, One Repository



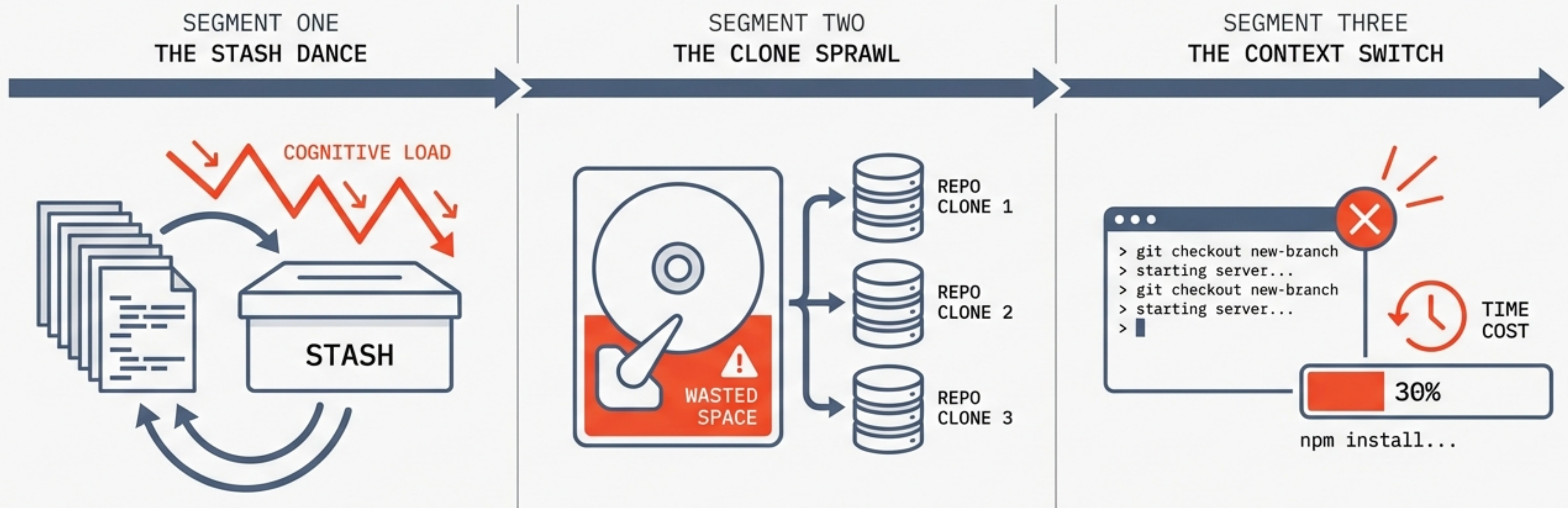
Eliminating context switching friction through Git's native architecture.

v3.72.1



The Friction of Modern Development

The classic Git workflow assumes a linear path: one repository, one directory, one task. Reality demands multitasking.

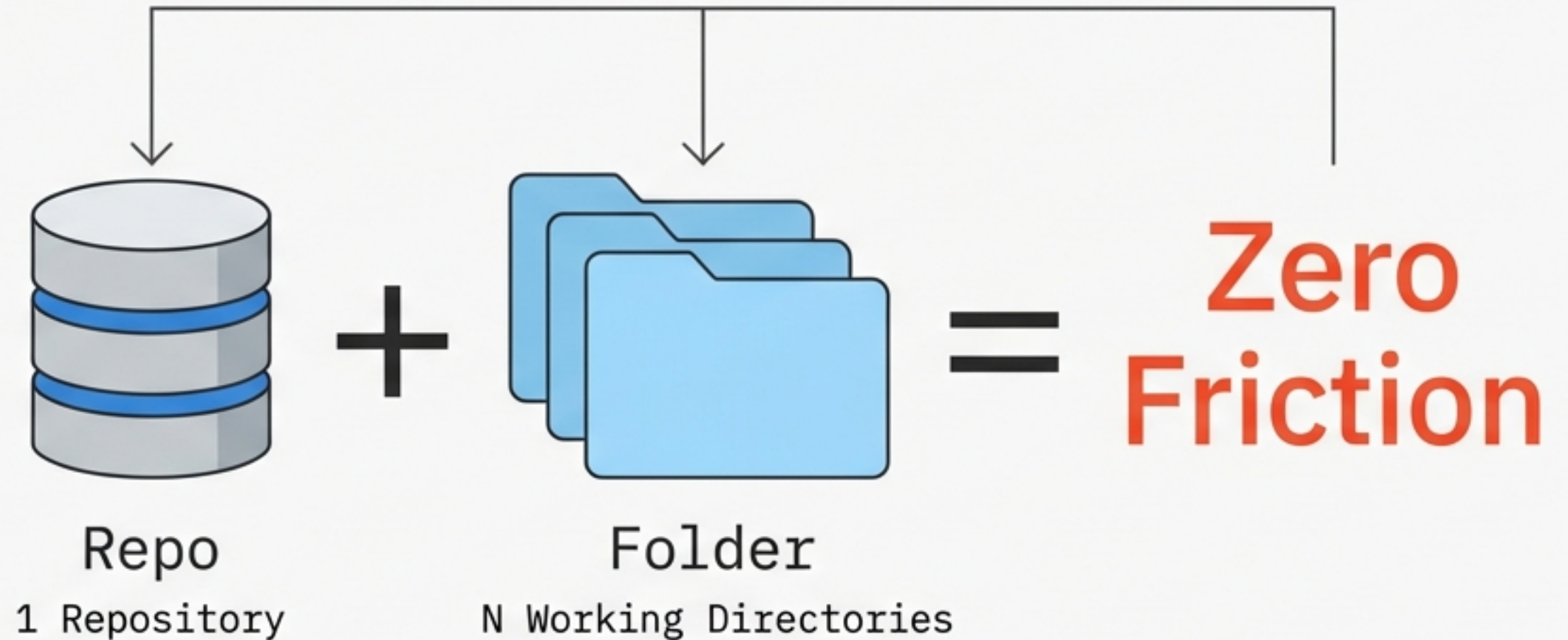


KEY INSIGHT: Current workarounds (Stashing/Cloning) are inefficient hacks. We need a native solution.

The Native Solution: Git Worktrees

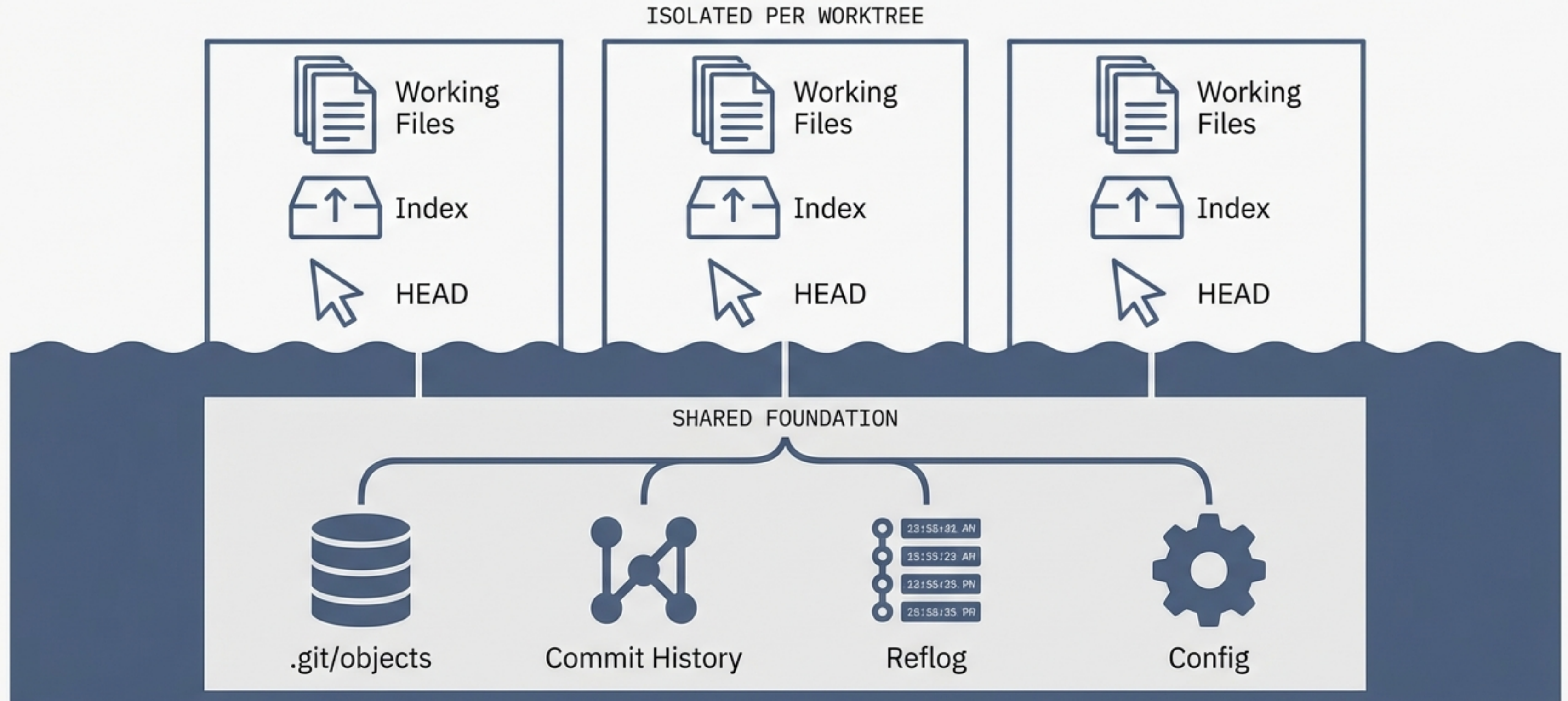
git worktree allows you to check out multiple branches simultaneously into separate directories while sharing a single object database.

- Work on a feature and a hotfix side-by-side.
- No stashing.
- No re-cloning.
- No synchronization.



Worktrees are not a hack, a plugin, or a special mode — they are a natural extension of Git's core architecture.

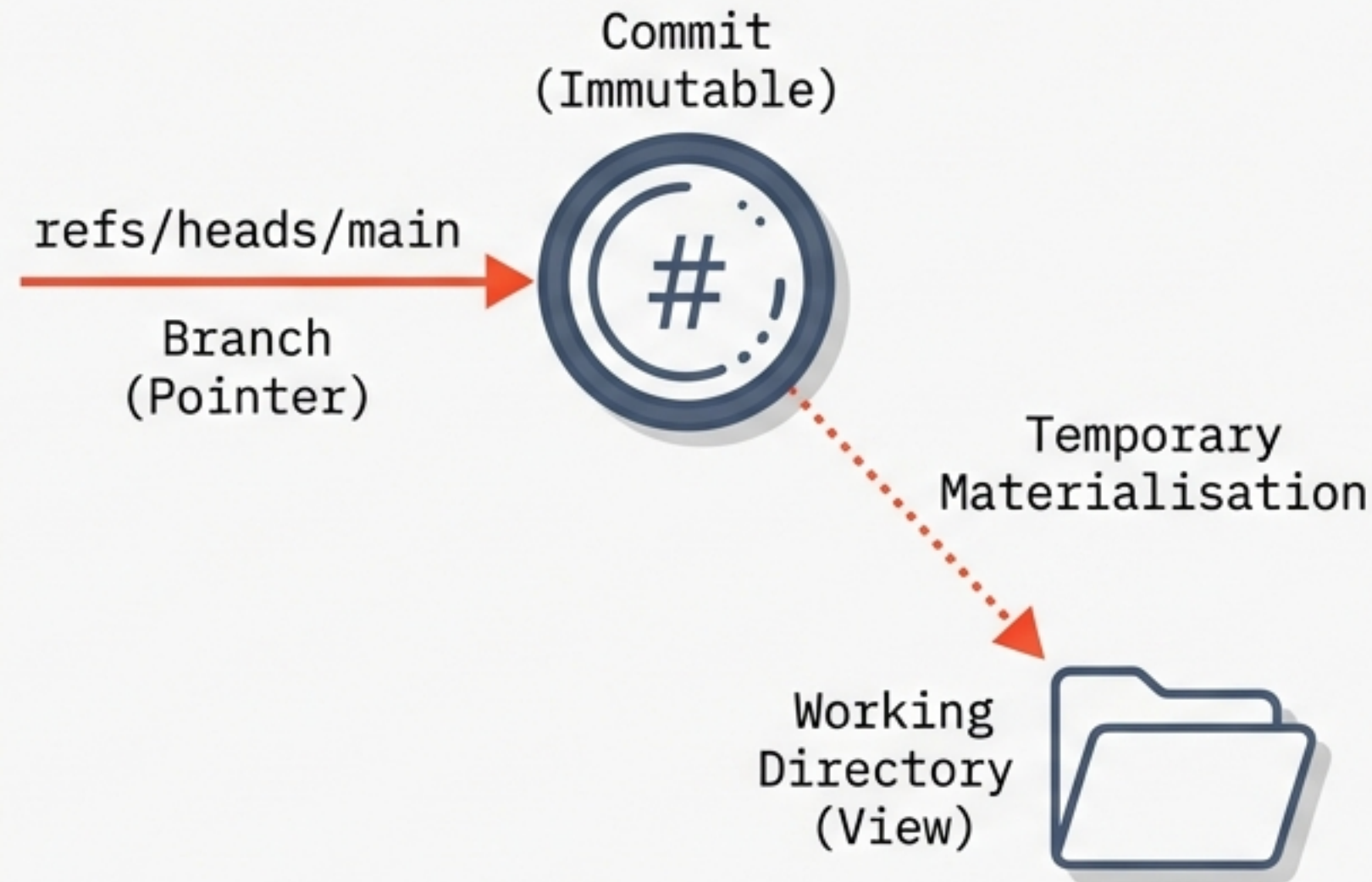
The Mental Model: Shared vs. Isolated



Multiple working directories, one immutable truth.

Why It Works: Git's Decoupled Design

Git is fundamentally content-addressed. History is distinct from the working state.



Key Concepts

- Commits do not belong to a folder.
- Branches are just movable pointers.
- A Working Directory is simply a view of a commit.

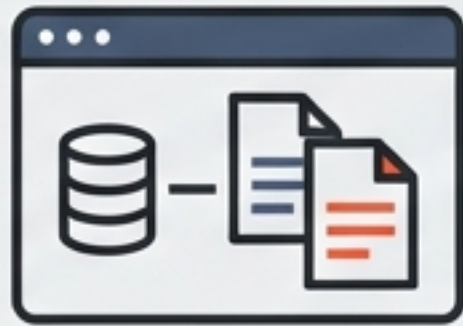
Because history $\neq$ working directory, Git naturally supports multiple views (worktrees) without data duplication.

Workflow: Creating a Worktree

A worktree creates a new folder that looks and acts like a repo, but is lightweight.

Check out existing branch

```
git worktree add ../folder-name  
branch-name
```



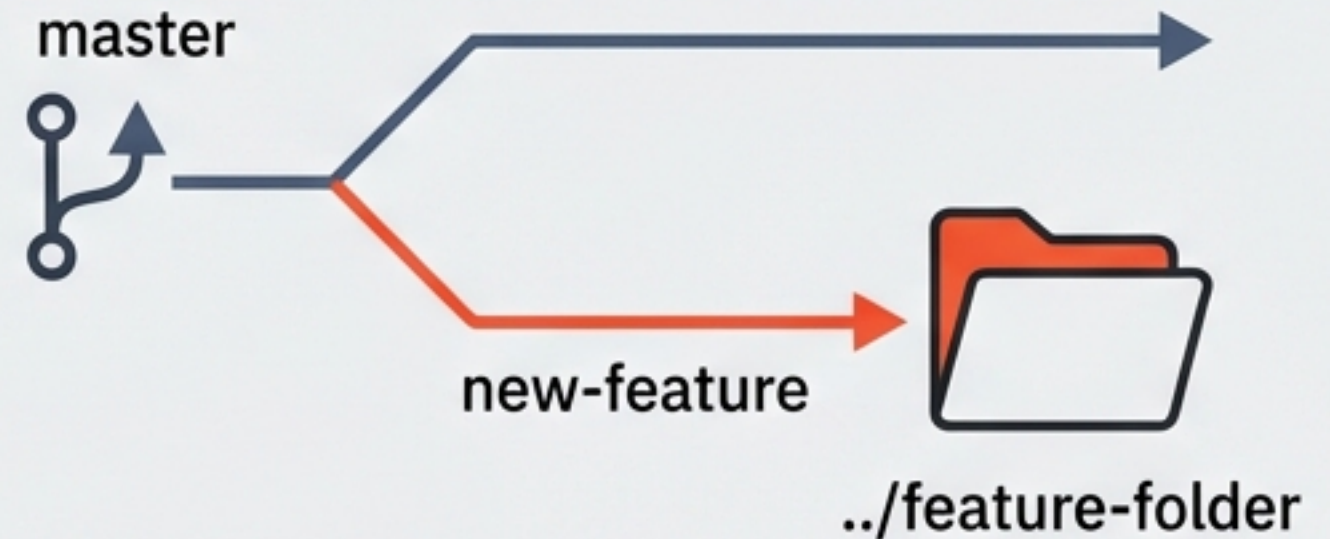
Main Repo



../folder-name

Create new branch & check out

```
git worktree add -b new-feature  
../feature-folder master
```



Workflow: Managing Contexts

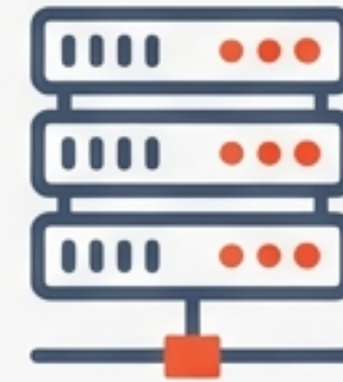
```
> git worktree list  
/Users/dev/repo (main)  
/Users/dev/repo-hotfix (hotfix/login-bug)  
/Users/dev/repo-review (pr/1024-update)
```

Throwaway Environments



Spin up for a quick PR review or bug triage, then delete. Keep your main environment pristine.

Long-Lived Environments



Dedicated worktrees for stable releases (e.g., release/v1.0) that need to be maintained alongside main development.

Safety Guarantees & The Golden Rule



A branch may only be checked out in one worktree at a time.

No Concurrent Rebases



No Conflicting States



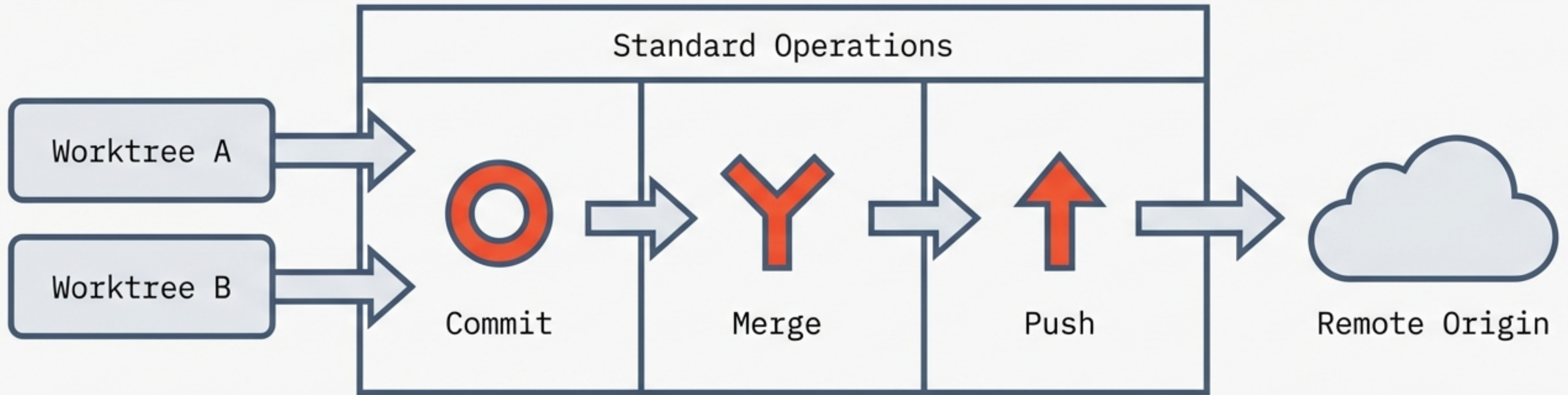
No Database Corruption



Exception: Detached HEAD worktrees (useful for code archaeology/debugging)

Day-to-Day Operations

From Git's perspective, nothing unusual is happening.



- Commit changes.
- Merge branches.
- Push to remotes.
- Open Pull Requests.

Housekeeping & Cleanup

The Clean Way

```
git worktree remove <path>
```



Deletes the directory and safely updates Git's internal administration files.

The Manual Fix

If you deleted the folder via 'rm -rf'...

```
git worktree prune
```



Cleans up invalid references in Git's internal list.

Comparative Analysis: Worktrees vs. Clones

Metric	Multiple Clones	Git Worktrees
Disk Usage	High (Duplicated .git)	Minimal (Shared .git)
Fetching	Repeated per repo	Shared (Fetch once)
Branch Safety	Manual (Risk)	Enforced (Locked)
Context Switching	High Friction	Zero Friction
Mental Overhead	High	Low

Verdict: Worktrees offer the isolation of clones without the resource cost.

When to Switch to Worktrees



Parallel Development

Working on a feature while a critical hotfix arrives.



Code Reviews

Checking out a colleague's PR locally without stashing your WIP.



Release Management

Maintaining long-lived branches (v1.0) alongside main.

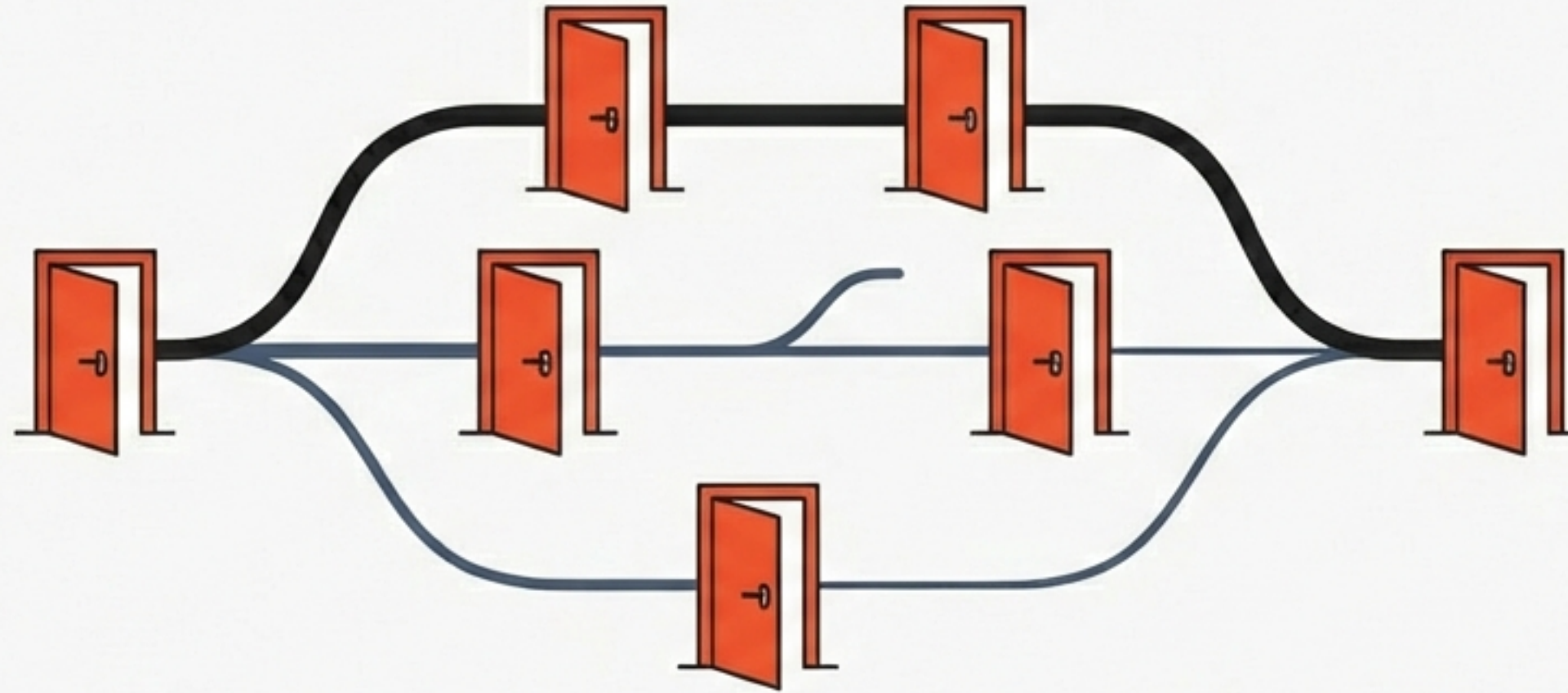


A/B Testing

Comparing two implementations side-by-side.

If you only ever work on one task at a time, worktrees are optional. But when complexity spikes, they are essential.

The Obvious Way to Use Git



git worktree does not change Git's model—it reveals it. By separating history from working state, we gain isolation without duplication.

Stop Stashing. Stop Cloning. Start Extending.

“Once you internalize this, worktrees stop feeling like a feature — they start feeling like the obvious way Git should be used.”